

WAOW / USER GUIDE

WAOW Admin Management Project

GitHub access, local setup, admin authentication, RBAC, API testing, and troubleshooting

Repository

<https://github.com/souksanDev1/waow-admin-task>

English Edition | Local Development Guide

Contents

- 01 What this project is
- 02 Requirements
- 03 Link GitHub
- 04 Clone the project
- 05 Run the project
- 06 How to use the app
- 07 API for testers / Postman
- 08 Push your changes to GitHub
- 09 Troubleshooting
- 10 Project map

Quick start

Two terminals required. Start the backend first, then start the frontend in a second terminal.

The admin API **shares the same PostgreSQL database** as the User API project (**waow-backend-task**) on port **5434** . Make sure that Postgres is already running (from the User API project or an equivalent Docker container).

```
# Terminal 1 – backend
cd backend
cp .env.example .env
pnpm install
pnpm db:migrate
pnpm db:seed
pnpm dev

# Terminal 2 – frontend
cd frontend
cp .env.example .env.local
```

```
pnpm install
pnpm dev
```

01 What this project is

WAOW Admin is a mini admin management system with JWT authentication and role-based access control (RBAC). It is made of two application layers and a shared PostgreSQL database:

- **Backend (backend/)**: Node.js + Express + PostgreSQL + Sequelize. JWT auth, dynamic roles/permissions, admin CRUD with soft delete, and read-only user listing under `/api/v1`.
- **Frontend (frontend/)**: Next.js App Router admin panel — login, dashboard, roles, admins, and users screens (pnpm, Tailwind, shadcn/ui, TanStack Query, Axios, React Hook Form + Zod).

App URL / Address	Value
Frontend	http://localhost:3001
Backend API	http://localhost:3002
Postgres (shared with User API)	localhost:5434

Default SUPER_ADMIN

- username: `superadmin`
- password: `123456`

Password is stored hashed. Change it in production.

Shared database. This project reuses the `users` table from the User API project and adds `roles` + `admins` tables on the same `waow_backend` database.

02 Requirements

Install these tools before cloning and running the project.

Tool	Why	Check
Git	Clone and push	<code>git --version</code>
Node.js 22+	Backend and frontend	<code>node -v</code>
pnpm	Package manager (both apps)	<code>pnpm -v</code>

Docker Desktop	PostgreSQL (shared User API DB)	<code>docker --version</code>
GitHub account	Clone / push	Sign in at github.com
GitHub CLI (optional)	Terminal login and repo creation	<code>gh --version</code>

Also required: the **User API Postgres** on port `5434` (from `waow-backend-task`), or start an equivalent container with DB name `waow_backend`.

03 Link GitHub

3.1 Sign in on the website

1. Open <https://github.com/login>.
2. Sign in, or create an account.
3. Open the project: [souksanDev1/waow-admin-task](https://github.com/souksanDev1/waow-admin-task).

3.2 Sign in from the terminal (optional)

```
gh auth login
```

Choose GitHub.com → HTTPS (or SSH) → follow the browser prompt. Then check your login:

```
gh auth status
```

3.3 HTTPS vs SSH

- **HTTPS:** easiest to start with. GitHub may ask for a Personal Access Token instead of a password when you push.
- **SSH:** generate a key, then add the public key in GitHub → Settings → SSH and GPG keys.

```
ssh-keygen -t ed25519 -C "your@email.com"
# then copy ~/.ssh/id_ed25519.pub into GitHub
```

04 Clone the project

Open a terminal and choose a working folder, for example Desktop:

```
cd ~/Desktop
git clone https://github.com/souksanDev1/waow-admin-task.git
```

```
cd waow-admin-task
```

SSH alternative:

```
git clone git@github.com:souksanDev1/waow-admin-task.git  
cd waow-admin-task
```

Expected project structure:

```
waow-admin-task/  
  backend/  
  frontend/  
  docs/  
  README.md  
  USER-GUIDE.md
```

Fork your own copy (optional)

1. On GitHub, click **Fork**.
2. Clone your fork instead of the original repository:

```
git clone https://github.com/YOUR_USERNAME/waow-admin-task.git
```

05 Run the project

Use two terminals. Start the backend first.

5.1 Shared database (User API Postgres)

If the User API project is already running its Docker Postgres on **5434**, you do not need another database.

Check:

```
docker ps | grep 5434
```

If nothing is listening on **5434**, start Postgres from the User API repo (**waow-backend-task**) first, or use this repo's optional **docker compose** on port **5436** and set **DB_PORT=5436** in **backend/.env** (standalone mode only).

Default configuration expects the **shared** database on **5434**.

5.2 Backend — API + migrations

```
cd backend
cp .env.example .env
pnpm install
pnpm db:migrate
pnpm db:seed
pnpm dev
```

Backend ready. When you see `Admin API listening on http://localhost:3002`, open:

<http://localhost:3002/api/v1/health>

It should return:

```
{ "error": false, "code": 0, "message": "Success", "data": { "ok": true } }
```

The users migration is **idempotent**: if `users` already exists (from the User API), it is skipped. Then `roles` and `admins` are created and seeded.

5.3 Frontend — web app

In a new terminal:

```
cd frontend
cp .env.example .env.local
pnpm install
pnpm dev
```

Frontend ready. When Next.js prints `Local: http://localhost:3001`, open that address in your browser.

5.4 Environment files

`backend/.env` (created from `.env.example`):

```
PORT=3002
NODE_ENV=development
JWT_SECRET=waow-admin-dev-secret-change-me
```

```
JWT_EXPIRES_IN=7d
DB_HOST=127.0.0.1
DB_PORT=5434
DB_NAME=waow_backend
DB_USER=postgres
DB_PASSWORD=postgres
```

frontend/.env.local :

```
NEXT_PUBLIC_API_URL=http://localhost:3002
```

Security note. Do not commit `.env` or `.env.local` to Git.

5.5 Useful scripts

Backend — run these from `backend/` :

Command	Purpose
<code>pnpm dev</code>	Start API with auto-reload
<code>pnpm db:migrate</code>	Create / update tables
<code>pnpm db:seed</code>	Seed SUPER_ADMIN / NORMAL + superadmin
<code>pnpm lint</code>	Lint backend

Frontend — run these from `frontend/` :

Command	Purpose
<code>pnpm dev</code>	Start web app on port 3001
<code>pnpm lint</code>	Lint frontend
<code>pnpm build</code>	Production build

Stop a server with `Ctrl + C` .

06 How to use the app

6.1 Sign in

1. Open <http://localhost:3001/login>.

2. Enter username `superadmin` and password `123456` .
3. Optionally use the **eye icon** to show/hide the password.
4. Click **Sign in**.
5. You land on the Dashboard.

Wrong credentials return an authentication error toast.

6.2 Dashboard

- Short welcome and links to Roles, Admins, and Users
- Sidebar: Dashboard, Roles (SUPER_ADMIN only), Admins, Users, Logout
- Brand: **Waow**

6.3 Roles (SUPER_ADMIN only)

1. Open **Roles** in the sidebar.
2. View existing roles (`SUPER_ADMIN` , `NORMAL` , and any custom roles).
3. **Create role** — set name and permissions for `admin_module` / `user_module` (`can_view` , `can_create` , `can_update` , `can_delete`).
4. **Edit / Delete** — system roles `SUPER_ADMIN` and `NORMAL` cannot be deleted (or renamed).

6.4 Admins

1. Open **Admins**.
2. With permission, **Create admin** (username, password, role).
3. **Edit** username / role / optional password.
4. **Delete** performs a **soft delete** (`is_deleted` + `deleted_at`). Soft-deleted admins cannot log in.
5. **Reset password** is available to `SUPER_ADMIN` only.

Permission flags on your role hide buttons you are not allowed to use.

6.5 Users (read-only)

1. Open **Users**.
2. See the list of users from the shared `users` table (same data as the User API project).
3. No create / update / delete in this admin task.

6.6 Sign out

Click **Logout** in the sidebar. You return to the login screen.

6.7 Roles & permissions summary

Role	Behavior
SUPER_ADMIN	Full access; role management; password reset

NORMAL	View admins/users only (no create/update/delete by default)
Custom roles	Dynamic JSON permissions per module

07 API — for testers / Postman

Base URL: `http://localhost:3002/api/v1`

All JSON responses use the same envelope:

```
{ "error": false, "code": 0, "message": "Success", "data": {} }
```

Failed example:

```
{ "error": true, "code": "ADMIN_ERR_001", "message": "Error message here", "data": {} }
```

Unknown fields in the body are rejected (`VALIDATION_ERR_UNKNOWN_FIELD`).

Method	Path	Auth
POST	/auth/login	none
GET / POST / PUT / DELETE	/roles	SUPER_ADMIN JWT
GET / POST / PUT / DELETE	/admins	JWT + admin_module permission
POST	/admins/:id/reset-password	SUPER_ADMIN JWT
GET	/users	JWT + user_module.can_view
GET	/health	none

Login

```
curl -s -X POST http://localhost:3002/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"superadmin","password":"123456"}'
```

Use `data.token` as a Bearer token for protected routes.

List admins

```
curl -s http://localhost:3002/api/v1/admins \  
-H "Authorization: Bearer YOUR_JWT"
```

List users (shared table)

```
curl -s http://localhost:3002/api/v1/users \  
-H "Authorization: Bearer YOUR_JWT"
```

Create role

```
curl -s -X POST http://localhost:3002/api/v1/roles \  
-H "Authorization: Bearer YOUR_JWT" \  
-H 'Content-Type: application/json' \  
-d '{  
  "name": "role1",  
  "permissions": {  
    "admin_module": {  
      "can_create": false,  
      "can_delete": true,  
      "can_update": true,  
      "can_view": true  
    },  
    "user_module": {  
      "can_create": true,  
      "can_delete": false,  
      "can_update": true,  
      "can_view": true  
    }  
  }  
'
```

More API / design detail: see `README.md` and `docs/superpowers/specs/` .

08 Push your changes to GitHub

```
git status
git add .
git commit -m "Describe your change."
git push
```

If this is a new repository of your own:

```
gh repo create waow-admin-task --public --source=. --remote=origin --push
```

09 Troubleshooting

Problem	What to try
Cannot connect to the Docker daemon	Open Docker Desktop, wait until it is running, then start the User API Postgres (or <code>docker compose up -d</code>).
port is already allocated / 5434 busy	Confirm which container owns 5434. This admin app should use that shared DB. Do not start a second conflicting mapping on the same port.
database "waow_backend" does not exist	Start the User API Postgres first; wait a few seconds; then <code>pnpm db:migrate</code> .
Frontend cannot call API	Confirm backend is on port 3002 and <code>NEXT_PUBLIC_API_URL=http://localhost:3002</code> is set in <code>frontend/.env.local</code> .
Login works but Users list empty	Shared DB has no rows yet — create users via the User API app, or insert demo rows.
<code>EADDRINUSE :::3002</code>	Another process is using 3002. Stop it, or change <code>PORT</code> in <code>backend/.env</code> and update <code>NEXT_PUBLIC_API_URL</code> .
Forbidden on Roles page	Only <code>SUPER_ADMIN</code> can manage roles.
Soft-deleted admin cannot log in	Expected. Soft delete sets <code>is_deleted</code> / <code>deleted_at</code> .

pnpm: command not found	Run corepack enable pnpm, or install pnpm from the official installation guide.
Page opens but login fails	Start backend before frontend; both services must be running; check health URL.

10 Project map

```

waow-admin-task/
|-- backend/      Express API, Sequelize, JWT + RBAC (pnpm)
|-- frontend/     Next.js admin panel (pnpm)
|-- docs/         Design + implementation plans
|-- README.md     Short setup
`-- USER-GUIDE.md

```

Frontend routes: `/login` , `/dashboard` , `/roles` , `/admins` , `/users` .

Backend API base: `/api/v1` .

Repository <https://github.com/souksanDev1/waow-admin-task>

END OF GUIDE